

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

SOFTWARE ENGINEERING – MINI PROJECT

Project: SmartSpend — Smart Expense & Budget Management Application

Test Plan

for SmartSpend (IEEE 829 Format)
Version 1.0

Team H5:

Sriram S – PES1UG24CS469

Sriram K – PES1UG24CS468

Srijan Kumar – PES1UG24CS467

Sri Pranav Gautam Buduguru – PES1UG24CS466

Table of Contents

1. Introduction
2. Test Items
3. Features to be Tested / Not Tested
4. Approach (Test Strategy)
5. Pass/Fail Criteria
 - 5.1 Security Validation
6. Test Environment
7. Test Deliverables
8. Traceability to SRS
9. Test Cases
10. Schedule & Responsibilities

1. Introduction

1.1 Purpose

This document is the Test Plan for SmartSpend, Team H5's expense and budget management application. It follows the structure of IEEE Std 829 (Standard for Software and System Test Documentation), specifying what will be tested, how, under what criteria, and with what evidence, so that testing activity can be verified against the SRS.

1.2 Scope

This plan covers unit, integration, system, and security-focused testing for all functional requirements (FR-001 through FR-021), non-functional requirements (NFR-001, NFR-002), and security requirements (SEC-001 through SEC-006) defined in the SmartSpend SRS, Version 1.0.

1.3 References

- SmartSpend Software Requirements Specification (SES-2026-H5_SRS.docx), Version 1.0.
- SmartSpend Software Architecture & Design Specification (SES-2026-H5_Architecture_Design_Spec.docx), Version 1.0.
- IEEE Std 829-2008 — IEEE Standard for Software and System Test Documentation.

2. Test Items

The following software items are subject to this test plan, each corresponding to a component defined in the Architecture & Design Specification:

Test Item	Description	Owner
Auth module	Registration, login, session issuance and validation	Member 1
Expense module	Expense create/edit/delete/categorize	Member 2
Budget module	Budget create/update, threshold recalculation	Member 3
Alert module	Threshold-based alert generation and display	Member 3
Analytics/Report module	Spend breakdown, trend charts, CSV export	Member 4
Admin module	Category management, aggregate analytics, RBAC enforcement	Member 4

3. Features to be Tested / Not Tested

3.1 Features to be Tested

- All functional requirements FR-001–FR-021 (registration, authentication, expense CRUD, budgeting, alerting, analytics/reporting, administration).
- Both non-functional requirements NFR-001 (response time) and NFR-002 (threshold recalculation latency).
- All security requirements SEC-001–SEC-006 (password hashing, RBAC, input validation, token validation, security logging, sensitive-field exposure).

3.2 Features Not to Be Tested (Out of Scope for This Phase)

- Real bank/payment integration — out of scope for the product itself (per SRS Section 2.6), so not applicable to this test plan.
- Multi-currency and localization behavior — explicitly out of scope per SRS Section 8.
- Load testing beyond the 50-concurrent-user baseline defined in NFR-001; large-scale load/stress testing is deferred to a future iteration.

4. Approach (Test Strategy)

Testing proceeds bottom-up across four levels, mirroring the layered architecture in the Architecture & Design Specification:

1. Unit testing — each Service-layer class (Auth, Expense, Budget, Alert, Report) is tested in isolation with the Repository layer mocked, using the framework-appropriate tool (pytest for Django, Jest for MERN).
2. Integration testing — API endpoints are tested against a real (test) database to verify the full request-to-persistence path, including validation and error handling.
3. System testing — end-to-end scenarios are executed through the actual UI or via API client (e.g., Postman), covering complete user journeys such as “register → add expense → receive alert.”
4. Security testing — dedicated authentication, authorization, and input-validation tests are executed as a distinct pass, detailed in Section 5.1.

Entry criteria: a feature is eligible for integration/system testing only once its unit tests pass and code has been merged via a reviewed pull request. Exit criteria: all Section 9 test cases for a feature must reach Pass status, with any Fail triaged and either fixed and retested, or explicitly deferred with team sign-off.

5. Pass/Fail Criteria

Overall Criteria

- A test case passes if the Actual Result matches the Expected Result exactly (see Section 9 table).
- A feature is considered test-complete when 100% of its Section 9 test cases pass, and code coverage for its Service-layer module reaches at least 70% line coverage.
- A Critical or High severity defect blocks sign-off for that feature; Medium/Low severity defects may be deferred with documented justification.

5.1 Security Validation

Security validation is treated as its own pass/fail gate, separate from general functional sign-off, because security failures (e.g., a broken RBAC check) are not acceptable to defer regardless of severity elsewhere in the system.

Security Objective (SRS 7.1)	Validation Activity	Pass Criterion
Confidentiality	Attempt to access another user's expenses/budgets via direct API call with a valid-but-different user's token	Request is rejected (403) for every attempted cross-user access

Security Objective (SRS 7.1)	Validation Activity	Pass Criterion
Confidentiality	Inspect all API responses for password hash or raw session secret leakage	No response body ever contains a password hash or the token signing secret
Integrity	Attempt to submit a negative or zero expense amount and an overlapping budget period directly via API (bypassing UI)	Both are rejected server-side (400/409), not merely blocked in the UI
Integrity	Attempt to invoke an Admin-only endpoint using a valid User-role token	Request is rejected with 403 for every Admin endpoint tested
Accountability	Trigger 3 consecutive failed logins and 1 unauthorized admin-endpoint call; inspect the security log	All 4 events are logged with timestamp and account identifier; no password is present in any log entry

A feature does not receive final sign-off until it passes both its Section 9 functional/non-functional test cases and the applicable rows above.

6. Test Environment

- Backend and database run in Docker containers matching the architecture defined in the Architecture & Design Specification, Section 3.
- A dedicated test database (isolated from any development/demo data) is used for integration and system testing, reset between test runs.
- CI pipeline (Jenkins or equivalent) runs the full unit test suite automatically on every pull request, per the course's mandatory CI/CD requirement.
- API-level system and security tests are executed using Postman (or an equivalent collection-based tool), with the collection checked into the repository.

7. Test Deliverables

- This Test Plan document.
- Test case execution records (Section 9 table, with Actual Result and Pass/Fail filled in during execution).
- Automated unit test reports and code coverage reports (per module).
- Security validation results (Section 5.1 table, completed).
- A defect log for any Fail results, with severity, root cause, and retest outcome.

8. Traceability to SRS

Every test case in Section 9 traces to exactly one SRS requirement (functional, non-functional, or security), and every High-priority requirement in the SRS has at least one corresponding test case here. This table is the test-side counterpart to the SRS's Appendix C (Requirement Traceability Matrix).

Test Case ID	SRS Requirement	Requirement Type
TC-FUN-01	FR-001	Functional
TC-FUN-02	FR-003	Functional
TC-FUN-03	FR-005	Functional
TC-FUN-04	FR-008	Functional
TC-FUN-05	FR-009 / FR-012	Functional
TC-FUN-06	FR-013 / FR-014	Functional
TC-FUN-07	FR-016	Functional
TC-FUN-08	FR-019 / FR-021	Functional
TC-NFR-01	FR-011	Non-Functional (performance)
TC-NFR-02	NFR-001	Non-Functional (performance)
TC-SEC-01	SEC-001	Security
TC-SEC-02	SEC-002	Security
TC-SEC-03	SEC-004	Security

9. Test Cases

The following 13 test cases (exceeding the required minimum of 7–10) cover functional, non-functional, and security requirements. Actual Result and Result columns are to be completed during test execution.

ID	Module	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-FUN-01	Auth	Register with a new, unique email	App running, DB empty of this email	1. Submit registration form 2. Verify account created	email=a@test.com, pwd=Passw0rd!	201 Created; user record persisted	<fill>	Pass/Fail
TC-FUN-02	Auth	Login with valid credentials	User already registered	1. Submit login 2. Inspect response	email=a@test.com, pwd=Passw0rd!	200 OK; session token returned	<fill>	Pass/Fail
TC-FUN-03	Expense	Add a valid expense	Logged in, category exists	1. Submit expense form	amount=250, category=Food, date=today	201 Created; expense visible in list	<fill>	Pass/Fail
TC-FUN-04	Expense	Reject zero-amount expense	Logged in	1. Submit expense with amount=0	amount=0, category=Food	400 Bad Request; validation error shown	<fill>	Pass/Fail
TC-FUN-05	Budget	Reject overlapping budget for same category/period	An active Food budget already exists for this month	1. Attempt to create a second Food budget for the same month	category=Food, period=this month	409 Conflict; second budget rejected	<fill>	Pass/Fail
TC-FUN-06	Alert	Generate alert at 80% threshold	Budget=1000 for Food, no prior spend	1. Add expenses totaling 850 2. Check alerts list	3 expenses summing to ₹850	Alert entry created and returned by GET /alerts	<fill>	Pass/Fail
TC-FUN-07	Analytics	Category-wise spend breakdown	User has expenses across 2+ categories	1. Call GET /analytics for current month	date range = current month	Response groups totals correctly by category	<fill>	Pass/Fail
TC-FUN-08	Admin	Admin disables a category	Logged in as Admin	1. Call disable-category endpoint 2. Verify category inactive	category_id=<valid>	Category is_active=false; existing expenses unaffected	<fill>	Pass/Fail
TC-NFR-01	Budget	Threshold recalculation completes within 1s	Active budget exists	1. Add an expense 2. Measure time to updated cumulative total	1 expense, amount=100	Recalculation reflected within 1 second (FR-011)	<fill>	Pass/Fail
TC-NFR-02	Full stack	Dashboard responds within 2s under load	50 concurrent simulated users	1. Run load test against /analytics	50 concurrent GET requests	95% of responses complete within 2 seconds (NFR-001)	<fill>	Pass/Fail

ID	Module	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-SEC-01	Auth	Password never stored in plaintext	New user registered	1. Inspect DB record directly	n/a	password_hash field is a salted hash, not the raw password	<fill>	Pass/Fail
TC-SEC-02	Admin API	User role blocked from Admin endpoint	Logged in as a regular User	1. Call POST /admin/categories with User token	User-role token	403 Forbidden returned	<fill>	Pass/Fail
TC-SEC-03	Auth	Expired token rejected on protected endpoint	A session token is manually expired/invalidated	1. Call GET /expenses with the expired token	expired token	401 Unauthorized returned	<fill>	Pass/Fail

10. Schedule & Responsibilities

Activity	Owner	Target
Unit tests for owned module	Each member (own module)	End of each sprint (see SRS Sprint Plan)
Integration tests (API + DB)	Module owner + peer review	Sprint 3–4
System tests (end-to-end)	All (shared)	Sprint 5
Security validation (Section 5.1)	All (shared, reviewed by all members)	Sprint 5
Final test report compilation	Testing/CI-CD secondary owner (Member 4)	Sprint 6